

Living Systems® Process Suite

Academy

Living Systems Process Suite Documentation

3.2
Tue Jan 12 2021

Whitestein Technologies AG | Hinterbergstrasse 20 | CH-6330 Cham
Tel +41 44-256-5000 | Fax +41 44-256-5001 | <http://www.whitestein.com>

Copyright © 2007-2021 Whitestein Technologies AG
All rights reserved.

Copyright © 2007-2021 Whitestein Technologies AG.

This document is part of the Living Systems® Process Suite product, and its use is governed by the corresponding license agreement. All rights reserved.

Whitestein Technologies, Living Systems, and the corresponding logos are registered trademarks of Whitestein Technologies AG. Java and all Java-based trademarks are trademarks of Oracle and/or its affiliates. Other company, product, or service names may be trademarks or service marks of their respective holders.

Contents

1	Academy	1
2	BPMN and LSPS	3
2.1	What is BPMN?	3
2.2	What is LSPS?	3
3	Working with Expressions	5
3.1	Evaluating Expressions	5
3.2	Creating Local Variables	8
3.2.1	Check Point	8
3.3	Functions	9
3.3.1	Extension Methods	9
3.4	Type Hierarchy and Casting	10
3.5	Related Documentation	10
4	Structure of GO-BPMN Process Models	11
4.1	Project	11
4.2	Module	12
4.3	Task: Create a Project	14
5	Business Data	15
5.1	Creating Records	15
5.2	Creating Relationships	16
5.3	Using Inheritance	17
5.4	Defining Methods	18
5.5	Persisting Data	18
5.6	Changes of a Data Model with Shared Records	19
5.7	Using Diagrams to Organize Records	19
5.8	Using Records from Other Definitions	21
5.9	Extending the Data Type Model	21

6	Interaction With Users	23
6.1	Getting Input as Part of a Process	23
6.1.1	Creating a Process with a User Task	24
6.1.2	Defining To-Do Properties	24
6.1.3	Running the Model	25
6.2	Getting Input Outside of a Process	25
6.2.1	Creating a Document Definition	25
6.2.2	Uploading and Accessing a Document	25
7	Goal Hierarchies	27
7.1	Defining Your Goals	27
7.2	Defining How to Achieve Your Goals	28
7.3	Making Sure Things are on Track	29
7.4	Execution	30
7.5	Summary	32
7.5.1	Pros	32
7.5.2	Cons	33
7.6	Documentation	33
8	Putting It Together	35
8.1	Prerequisites	35
8.2	Registration Data	36
8.3	Client and Employee User	37
8.4	Registration and Approval	37
8.4.1	Registration Process	38
8.4.2	Approval Process	39
8.5	Running the Model	40
9	Customizing the LSPS Application	43
9.1	Generating the LSPS Application	43

Chapter 1

Academy

This guide leads you through the main features of LSPS: Each chapter is generally self-contained and focused on a particular part of the product. Where applicable, we provide links to further information.

After you get through the Academy guide, you should have a good overview of LSPS features and be able to work independently with the help of reference documentation.

Chapter 2

BPMN and LSPS

2.1 What is BPMN?

Originally IT infrastructure relied on a model where humans drove the business process based on the data status: only the business data was stored leaving any knowledge on how to work with the data with humans: for example, if a client wanted to open a bank account, the client filled out an application form and a bank employee performed and overlooked the processing of the application, which could lead to errors.

BPMN is a standard that defines how to capture and execute such processing in a set of steps that achieve your goal called a *business process*. These steps can be performed by people or automatically. Business processes are represented visually and executed in a standardized way: the visual representation provides common grounds for business people on the one side and programmers on the other: unlike code, the business people can understand the visualization, and check and analyze the process.

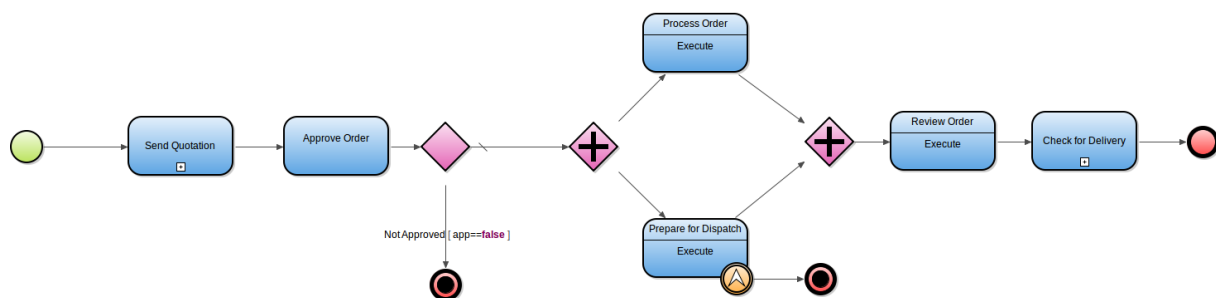
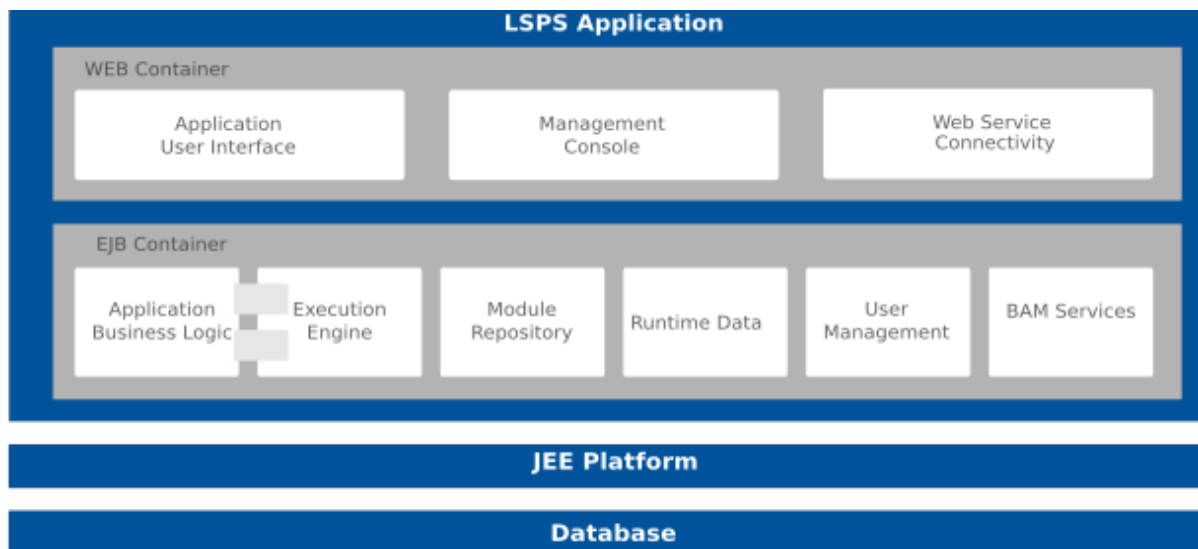


Figure 2.1 Process in BPMN notation

2.2 What is LSPS?

LSPS is a set of tools that allows you to design and execute BPMN processes. Built up based on BPMN, LSPS expands its capabilities: it adds support for GO-BPMN, conservative goal-focused processes; role-based access control; front-end page definitions, data type models, and much more to your models so you can build an entire application for your business.

The core of LSPS is the *LSPS Application*, a standard JEE enterprise application. The application stores your models, executes them, manages users, distributes work, provides access to the data from web applications, communicates with external systems and users, etc.



To create the models of your business processes for the *LSPS Application*, you will use the **Process Design Suite**. Apart from being a development tool for models, it is also a thick client for management and administration of the LSPS Application resources.

When it comes to interactions with the LSPS Application, you have also other options than PDS:

- If you are a process participant, use the **Application User Interface**, a web application that serves to get input for processes from human users.
- If you are an administrator, use the cli console or **Management Console**, a web application that serves to manage the resources of the LSPS Application, such as, models, model instances, users, etc.
- If you need to integrate with other systems or tools, use the web services API.

Depending on your needs, consider to install the following:

- **Enterprise Edition** comes with the Process Design Suite and LSPS SDK, which allows you to generate the LSPS Application EAR with the source code of the *Application User Interface*.
- **Runtime Suite** contains the LSPS Server EAR, CLI console, and database migration scripts. It is intended as an administration tool bundle.

To install the enterprise edition, follow the instructions in the [Installation Guide](#).

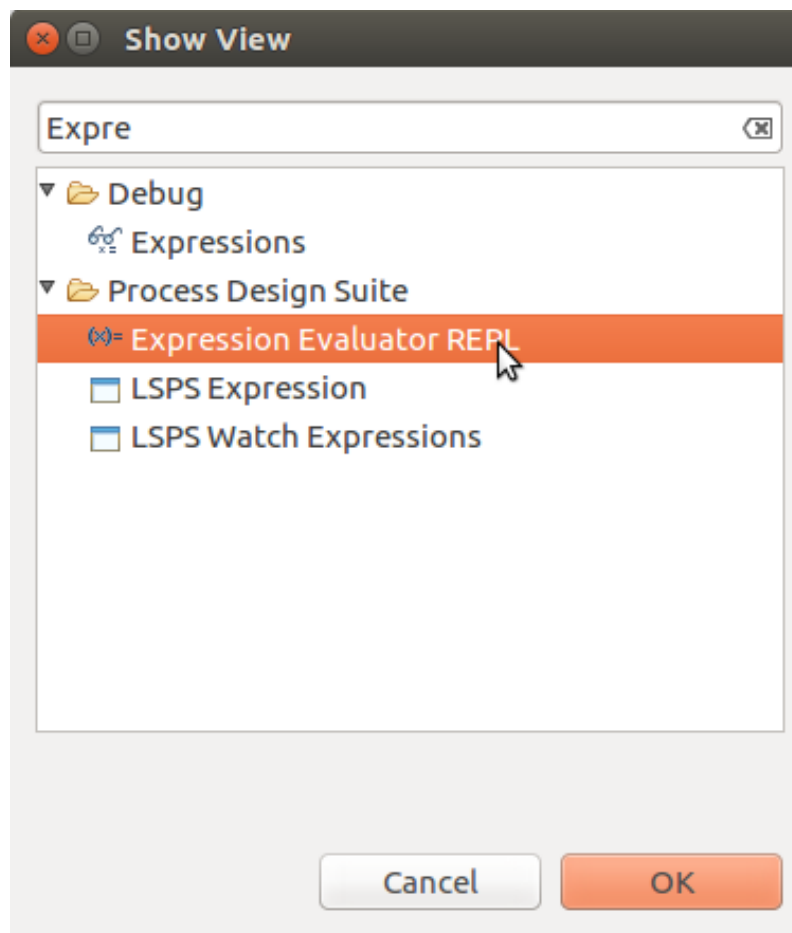
Chapter 3

Working with Expressions

Properties, conditions, assignments, etc. are all defined as expressions of the LSPS Expression Language. Expressions use operators, literals, keywords, and other constructs of the Expression Language, that end up returning a value of a type. Also literals are of a type, which does not change (unless cast but we will get to that later).

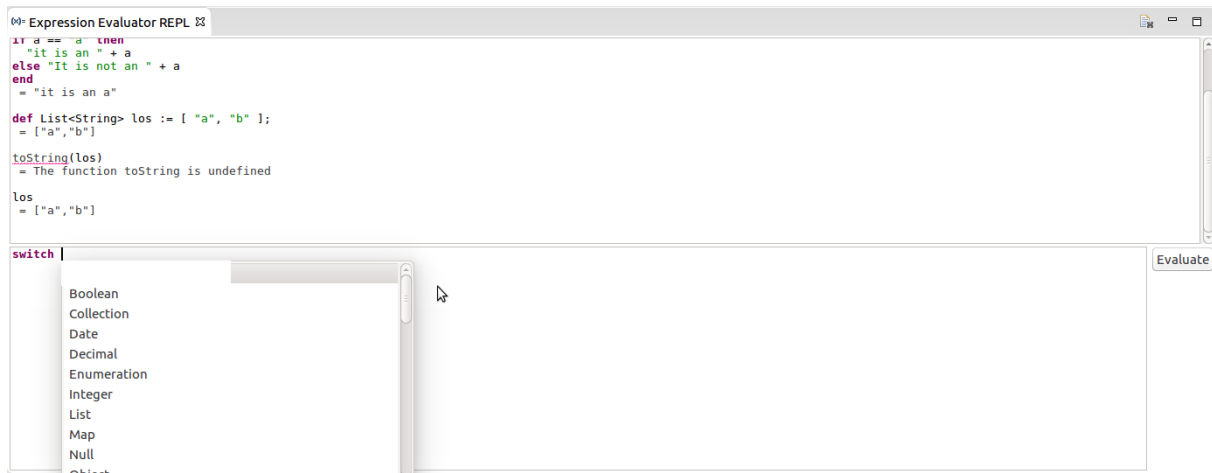
3.1 Evaluating Expressions

You can test expressions in the **Expression Evaluator REPL** view. By default, the view is not displayed: To display it, go to **Window > Show View > Other** and in the displayed dialog search for the view.



To test an expression and display its return value, enter the expression into the view and click **Evaluate** or press CTRL+Enter (Enter will make a new line in your expression). Note that you can list the history of expression by pressing the arrow-up key and display auto-completion options by pressing CTRL+Space.

The REPL view supports calls to Standard Library resources, such as functions, data types, etc.



Note: When evaluating your expression in the Expression Evaluator REPL view, the expressions are evaluated in a "dummy context" outside of any model: under normal circumstance, on runtime, expressions are evaluated in a context of their model instance: so they can access data in their parent contexts. Contexts are represented by Instances of name-space elements: These elements include Modules, Processes, Plans, and sub-Processes. To fully understand when a context is created and what data it holds, refer to [namespaces and contexts](#).

Let's try it out:

Enter the following into the Expression Evaluator REPL and try to predict the return value:

- First, let's try some arithmetics:

```
- 1 + 1
- 1 + 1.12
- 1/1.1
- 1%1.1
- 3**3
```

- Let's try some concatenation:

```
- "Hello, " + "world!"
- "Hello, number " + 1
- 1 + "hello" This results in an invalid expression since the + operator is interpreted as addition, not concatenation.
```

- Let's try some comparing:

```
- "Anne" < "James"
Strings are compared lexicographically: A appears before J in alphabet.
- "James" like "J*"
- "James" like "J???s"
- "James" like "Jo*"
```

- `1 == 1,00`
- `"Bob" <=> "Anne"`
- `"Bob" <=> "John"`
- `"Bob" <=> "Bob"`
- `d'2015-12-24 20:00:00.000' < d'2017-12-24 20:00:00.000'`

- And now let's do some logic:

- `true and false`
- `true and true`
- `false and false`
- `true or false`
- `false or true`
- `true or true`
- `true xor false`
- `false xor true`
- `true xor true`
- `true xor true`
- `!true`
- `!false`

- Now, chain multiple expressions:

- `"This will not return!"; "Hello, " + #10 + "world!"`
Note that only the value of the last expression is returned.

- Create Collections with items of some data type:

- Sets: `{1, 2, 3}` and `{1, 1, 2, 3}`
- Lists: `[1, 1, 2, 3]`

Sets cannot contain items of the same value while Lists can.

Mind that Collections are immutable: when you decide to change a List you need to create it anew.

- `[ 1 -> "Sunday", 2 -> "Monday"]` is a **Map**. In this case its of type `Map<Integer, String>`.

- Evaluate the closure `{x:Integer -> "Integer" + x}`

The closure has an Integer input parameter called `x` in the closure contexts: the closure a String, hence it is of type `{ Integer : String }`

Important: It is recommended to always explicitly define the type of the closure inputs, in the example, we explicitly define that `x` is an *Integer* in `x:Integer`.

We have been through some basic data types but there are other data types to explore and we will get gradually to all of them. Let's take a sneak peek and evaluate the following:

- `type(Integer)` returns the Integer type.
- `Null` is a special data type with the sole value `null`: all other types are super types of the Null type so an expression of any data type can all have the value `null`.
- `&var` is a reference to a variable.

Note: We omitted Records, which are complex data structures, and the related data types one of the reasons being that it is not possible to create a new Record type with the Expression Language since Records are modelled in diagrams similarly to processes.

3.2 Creating Local Variables

In expressions, you can create local variables using the syntax `def <TYPE> <VARIABLE_NAME>`.

Try this in REPL view:

```
def Boolean boolVar;
```

Though the Boolean variable exists now, it has no value: its value is `null` of type `Null` as we mentioned above. To assign a value to a variable, use the assignment operator `:=`.

```
boolVar := true
```

Local variables cannot be accessed from outside of their scope, expression block. An expression block is the expression, a loop, an if block, and an explicit block between the keywords `begin` and `end`.

Evaluate the following:

- Create a block with a local variable:

```
begin
  def Boolean boolVar := true
end
```

- Call the variable:

```
boolVar
```

If you need a variable that you can access from throughout the model or a particular namespace, use either global or local variables: however, these are part of modeling and we will deal with them in later courses on model resources.

3.2.1 Check Point

1. Create a local variable called *closureVar* of type `Closure`, which takes a list of integers as its input parameter and returns a `String`.
2. Assign the variable a closure that concatenates the input list into a single `String` and returns the `String`.
3. Create a list with integers 1-100 and feed it to the closure variable.

The entire expression should look something like this:

```
def {List<Integer>:String} closureVar;
closureVar := { myIntList:List<Integer> ->
  def String output;
  foreach Integer i in myIntList do
    output := output + i + ", "
  end;
  output
};
def List<Integer> myIntList := 1..1000;
closureVar(myIntList)
```

3.3 Functions

You can consider functions special types of closures: they take arguments of certain types and return a value of a certain type, plus they can be used whenever you would use a closure: They are sort of closure constants.

To call functions from an expression, use the syntax `<FUNCTION_CALL> (<PARAMETER_1>, <PARAMETER_2>)`.

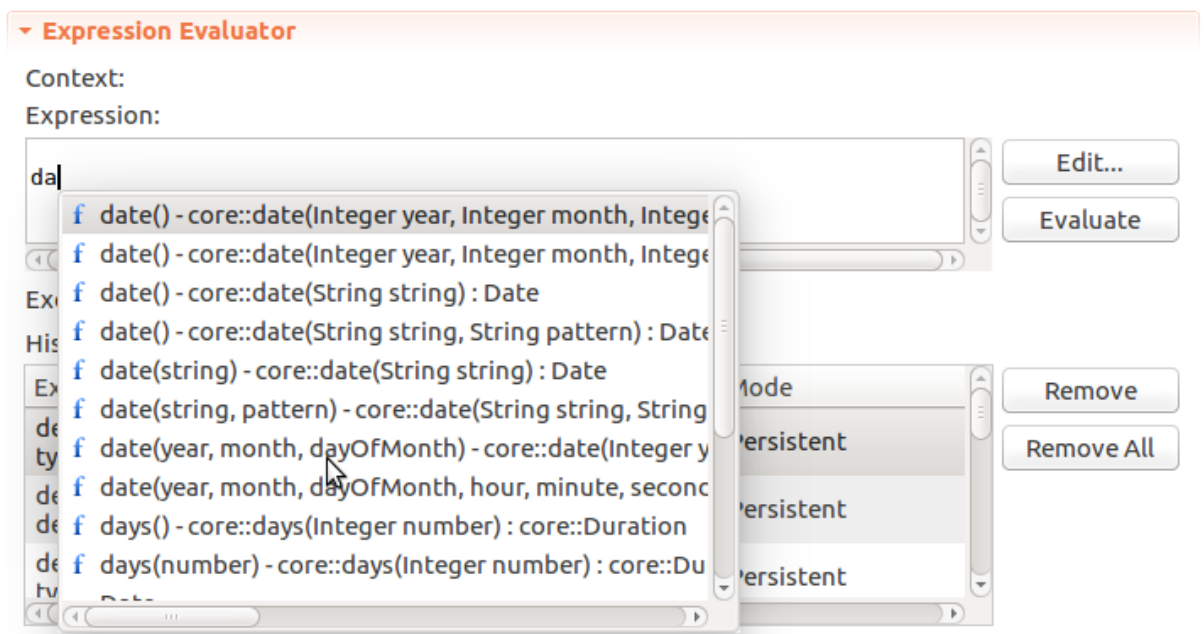
You cannot declare new functions in any expression but only in a dedicated function definition file.

Luckily, there are plenty of functions available in the Standard Library which is automatically available in your Module.

Let's try it out:

Creating a date value like `d'yyyy-MM-dd HH:mm:ss.SSS'` is quite cumbersome: luckily there is a function that will make your life much easier: the `date()` function.

In the REPL view, start typing `date` and press `Ctrl + space` to display the auto-completion dialog with the date functions. Click on the one you like and define its parameters so it returns the correct date value.



3.3.1 Extension Methods

If a function has a required parameter, it can be defined as an extension method: this means that you can call the function with the syntax:

```
<ParameterObject>.<function_name>()
```

Hence, instead of writing `toString(myInteger)`, you can write `myInteger.toString()`

To find out, if a function of the Standard Library has its extension method, go to its declaration and look for `@ExtensionMethod` annotation or **Extension method** flag, depending on the function resource.

Here, we check if the function definition has an Extension method flag and then rewrite the function call to an equivalent extension method call:

```
def List<Integer> myList := collect(1..10, {x:Integer -> x + 2 });
(1..10).collect({x:Integer -> x + 2 })
```

3.4 Type Hierarchy and Casting

Data types are arranged in a hierarchy which restricts relationships between data types. A type can be a subtype or a supertype of another type: If a type is a subtype of another type, it can be always used instead of the supertype, but not vice versa. Since the typing is so strict it prevents you from using incorrect types; for example, you cannot accidentally pass a Date value to a String variable unless you explicitly cast it to a String.

For the types of the language, the most generic data type is the Object type: all data types are subtypes of the Object type: when a variable is of the Object type, you can assign it a value of any type.

In a complex type, one is a subtype of the other if their inner members are each other's subtypes: `Map<KA, VA>` is subtype of `Map<KB, VB>`, if KA is a subtype of KB and VA is a subtype of VB.

Let's try it out:

In the Expression Evaluator of your model instance, create an Object variable, for example, `def Object myObject`, and assign it a value of any type. To check the type of value the object holds, run `typeof(myObject)`. If you evaluate `def Object myObject := [1,2,3]; typeof(myObject)`, the expression will return the type `List<Integer>`

Otherwise, the data type structure of the built-in data types is pretty flat: there are only two other relationships:

- The Decimal type and Integer type with the Decimal type being the super type: Wherever you can use a Decimal value, you can use an Integer value just as well.
- The Collection type and, the Set and List types: Collection is however abstract; it serves to allow you to decide whether something will be a Set or List later during execution. Let's try it out; Evaluate the following:
 - `def Collection<Object> myObject := [1,2,3]; typeof(myObject)`
 - `def Collection<Object> myObject := {1,2,3}; typeof(myObject)`

If you want to change the value type, typically from a supertype to a subtype, you can use the build-in cast mechanism. Let's try it out: In the Expression Evaluator, evaluate the following expressions:

- `1 as Decimal`
- `1.00 as Integer`
- `1.12 as Integer`

3.5 Related Documentation

For details on features of the Expression Language, refer to the [Expression Language Guide](#). As a reference, consider using the [Expression Language Reference Card](#).

Chapter 4

Structure of GO-BPMN Process Models

A *Process Model* or *Model* is a sum of an executable module with all its resources including its imported modules. It is not represented explicitly since any executable module can become a model but also be imported into other modules.

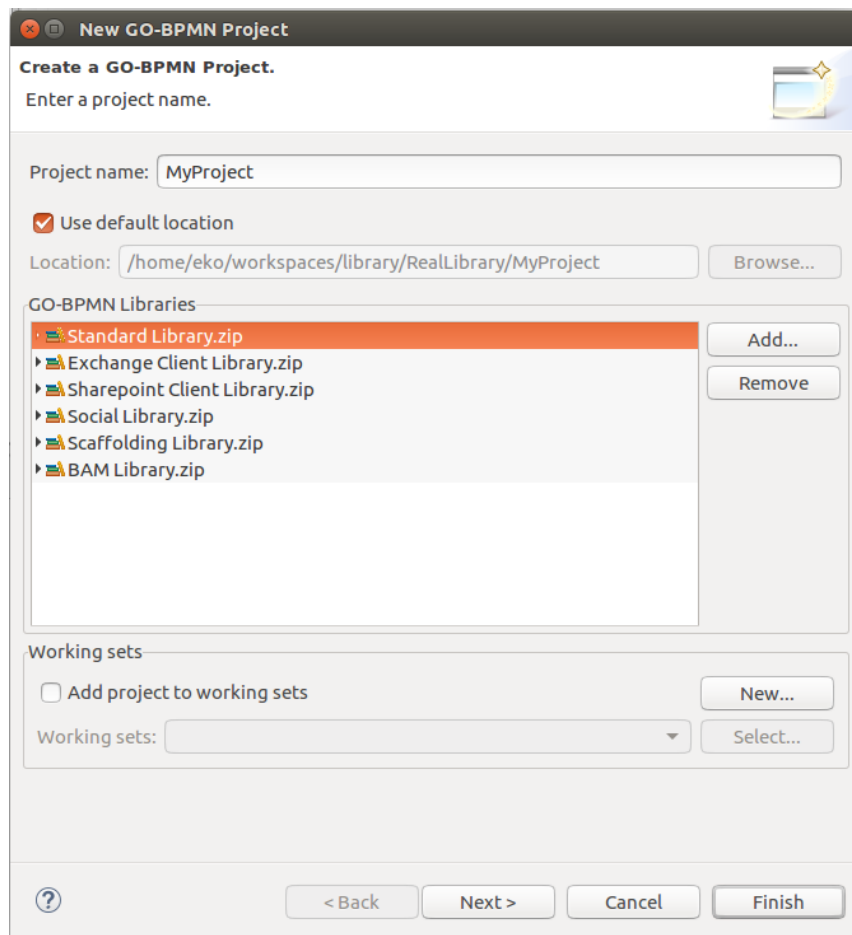
A GO-BPMN Project is by default created with the Standard Library and the modules import the library by default as well. On design time, modules can be created only in Projects. To further organize the content of projects and modules, you can create folders in Projects and Module: these are not uploaded to the server.

4.1 Project

A Project is a directory and serves only for organization of your sources: they are not influence semantics of your model and are not uploaded to the LSPS Server. If you decide to create other projects to organize your resource, by default, you will not be able to use resources from one project in another: to be able to access stuff from another Project, you need to explicitly enable the access by referencing it. We will get to referencing later. Let us create and examine a GO-BPMN project now.

Let's create a GO-BPMN project:

1. In the GO-BPMN Explorer, right-click into empty space and go to **New > GO-BPMN Project**. In the displayed dialog below the Location field, there is the list of libraries that will be imported into the project. The only one that is required is the Standard Library: the other libraries provide additional resources you might not need.



2. Name the project and click **Finish**.

4.2 Module

Modules are similar to java jar files. They follow a strict structure and can have dependencies, called imports. A Module must be in a Project and all resources must be encapsulated in a Module. Note that what gets uploaded to the server is a Module: a Project exist merely on design time for your convenience.

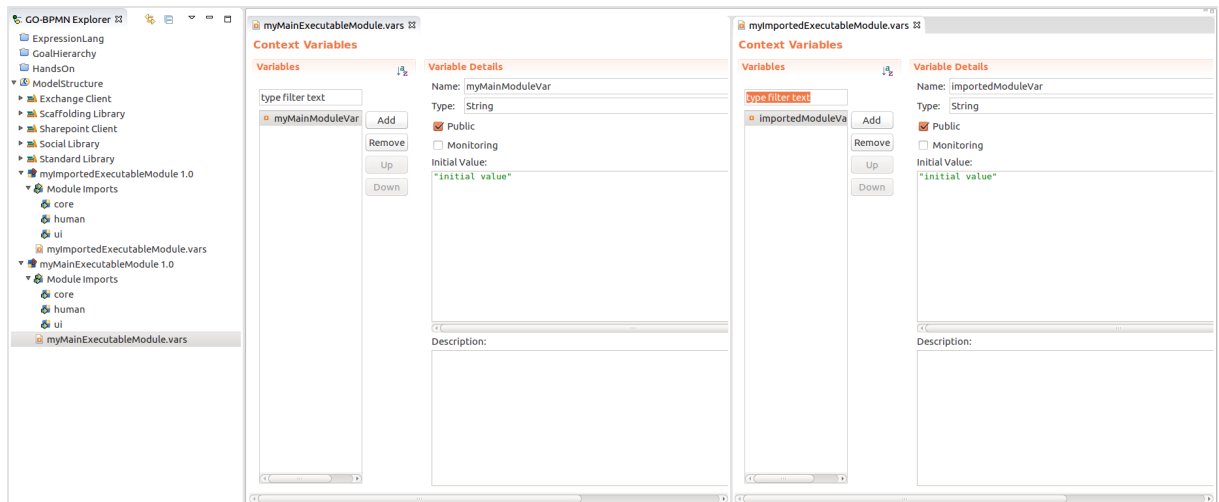
A Module holds most resources: processes, variables, custom data types, queries, functions, etc. Ideally it should be a reusable independent collection of resources. When you upload a module nothing much happens: the resources are simply uploaded to a repository on your server. However, if the Module is executable, you can instantiate it once it was uploaded. Marking a module as executable is basically like marking a method as `main`: it signalizes that this module is where execution can start. Unlike with `main`, you can have multiple executable Modules in one Module. How so? You can import them into other Modules: when you run your executable module, a model instance is created. Inside the model instance, an instance of the executable module and any other imported modules are created and run as well.

Note that you can create further folders in Projects and Module to organize your resources. Their behavior is the same as the Projects behavior: on upload, they are ignored.

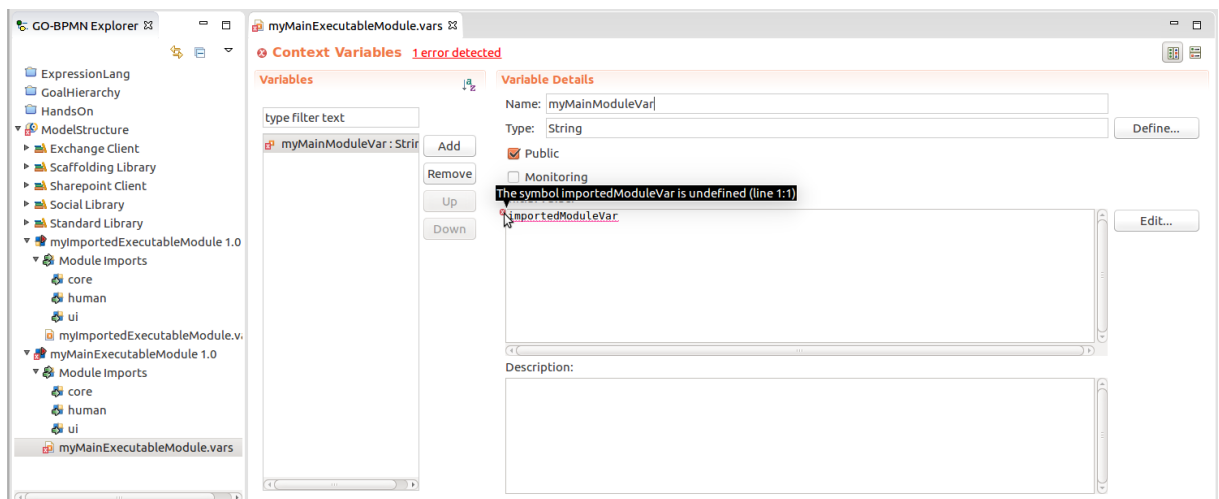
Let's create an executable module:

1. Now create a module in the project: right-click the project, go to `New > GO-BPMN Module`. Name it `myMainExecutableModule`. Make sure to leave the executable flag selected.

2. Create another module in the project: Name it *myImportedExecutableModule*. Make sure to leave the executable flag selected.
3. Let's create in both modules some global variables: right-click the module name, go to **New > Variable Definition** and then click **OK**. In the open editor, click **Add** and on the right, define a variable. These will serve us to check what we can access in the modules.

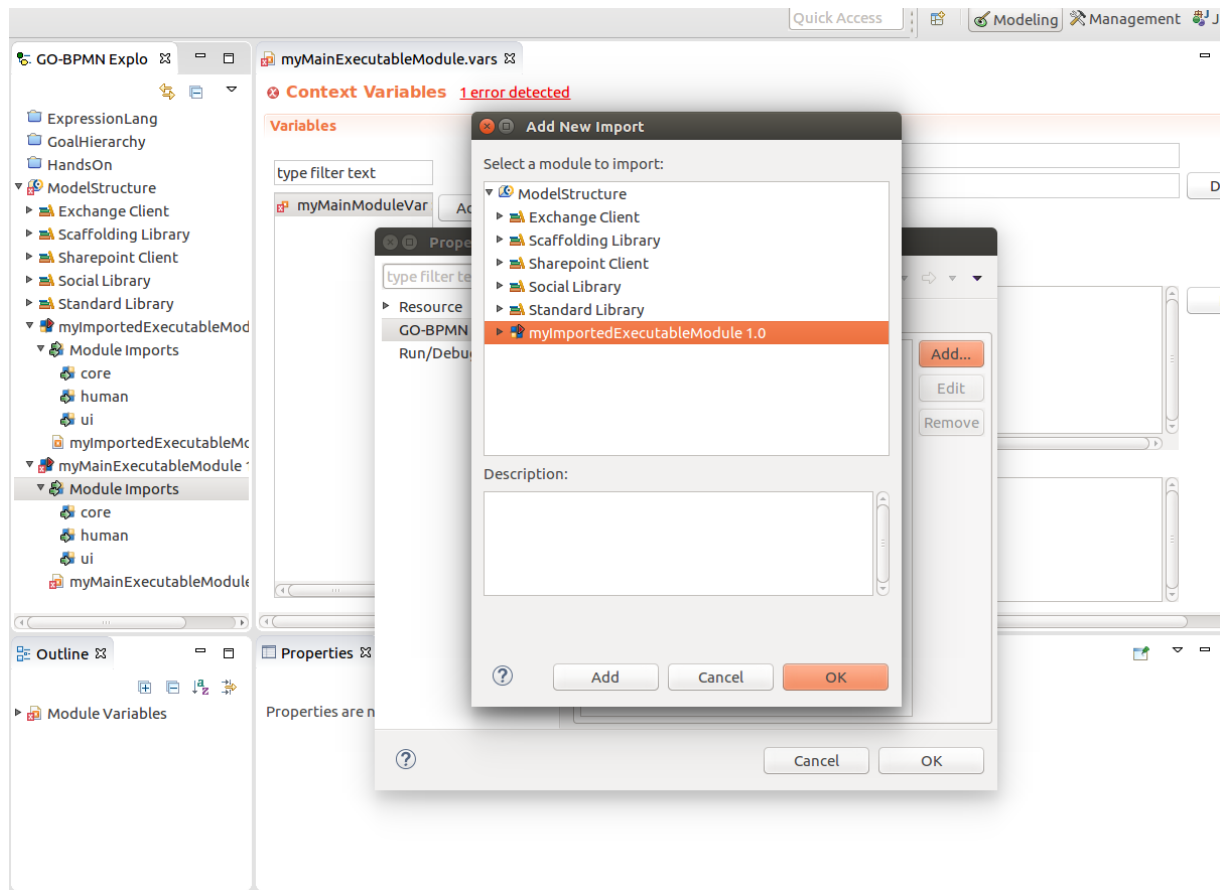


4. In a global variable in *myMainModule*, try to use the variable from the *importedModule*: you will end up with a validation error, that no such entity is available.



To solve this problem, import the importedModule into myMainExecutableModule.

5. In the GO-BPMN Explorer, double-click the Module Imports node in myMainExecutableModule.
6. In the dialog box, click Add and double-click myImportedModule.



7. Watch your reference problem go away: the resources of the importedModule are now available in the myMainModule.
8. You can visualize module dependencies in the Module Dependency View: on the main menu, click **Window** > **Show View** > **Module Dependency View**.

4.3 Task: Create a Project

Create another GO-BPMN Project with a Module and import the *myMainExecutable* module into your new module: you will need to *reference the project*.

Chapter 5

Business Data

Every company works with data that have some structure. Let's take invoices for example: as a bare minimum they have an ID number and a date. The rest of the data is where it gets more complicated: the bill-to data and the list of purchased items are further data structures and they depend on each other. Such data structures are reflected as custom complex data types, called Records.

You create them in dedicated data type definitions in a visual editor which makes the process easy and intuitive: individual data structures, like invoices, items, people, etc. are represented by Records.

This chapter provides only a brief introduction into data types and data type modeling. Note that to create both sophisticated and efficient data models, you will need more than being able to work with LSPS: make sure to analyze your data model properly before you design it to prevent performance and maintenance issues in the future. More detailed information on data types is available in [the Data Type Model section of the GO-BPMN Modeling Guide](#).

5.1 Creating Records

We will create a Record called *Invoice* and another Record called *BillTo*. To include data about the Invoice in the BillTo record, we will add an invoice field to the BillTo.

Let's do that:

1. First you need to create a *data type definition* in your Module. This is a file that will contain the data type model with the Records:

Right-click the Module and go to **New > Data Type Definition**. Provide the name and click **Finish**.

The name of the file is not important since the server does not care for file name: you can have your data model in multiple *data type definitions* and they will be still considered one data model: what "separates" your data models is a module (the namespace is defined by the name of the module), not files.

2. You can see the definition file in your Module in the GO-BPMN Explorer and it should open in the data-model editor: the white empty space of the canvas in the editor is actually a diagram, which is a visualization unit: it displays the content or part of the content of the data type definition. The actual content of the definition file is in the Outline view. We will return to diagrams later.

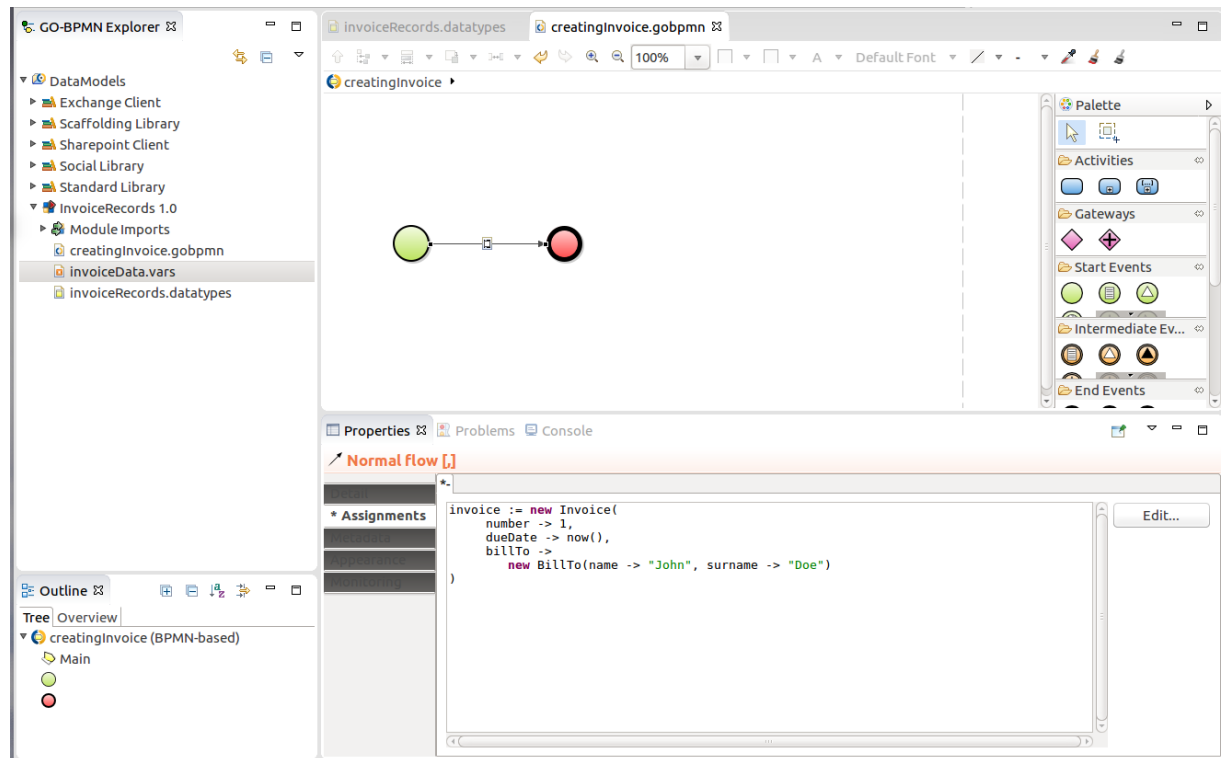
In the diagram, create the Invoice Records with the fields *number* of type Integer and *dueDate* of type Date.

3. Create another record, the **BillTo** record, with the fields *name* and *surname* of type String.

To quicken the modeling process, insert a new Field with the **Insert** key and move to the field type with the tabulator key. You can copy and paste the Records as well as their Fields.

4. To add the bill-to data to the Invoice, create the *billTo* Field of type **BillTo** in the Invoice.

Now you can create instance of the Invoice record with the data about the invoice stored in its fields, for example, in a process on a Flow assignment: the expression will be executed when a token is passed from the Start Event to the Flow. We store the Record instance in the global variable `invoice`. The data of the global variable can be accessed from anywhere within the module with the expression `invoice.billTo.name`.



5.2 Creating Relationships

You have probably spotted a problem in the data model: What if the same person places multiple orders? You will end up creating multiple invoices with the same Person data again and again. If the data of the person change, you will need to update them in every single Invoice instance. We can solve this by associating the two Records with a relationship: the Invoice record will be only *related* to a BillTo record and so multiple Invoices can be related to the same BillTo person:

1. Delete the billTo field.
2. Grab the quicklinker on the Invoice record and pull it to the BillTo record. Then define the properties of the relationship.

Note that we have named both relationship ends so we can navigate from the Invoice to BillTo and vice versa (that means, we can now write expressions like `myInvoiceInstance.billTo` and `myBillTo.invoices` to access the related record instance).

Also, we have set the multiplicity on the *invoices* end to Set: this allows us to associate multiple invoices with a single BillTo. Here is an example:

```

currentInvoice_1 := new Invoice(
    number > 1,
    dueDate > now()
);
currentInvoice_2 := new Invoice(
    number > 1,
    dueDate > now()
);
//john is a global variable of type BillTo: john := new BillTo(
    name > "John",
    surname > "Doe",
    invoices > {currentInvoice_1, currentInvoice_2 }
)

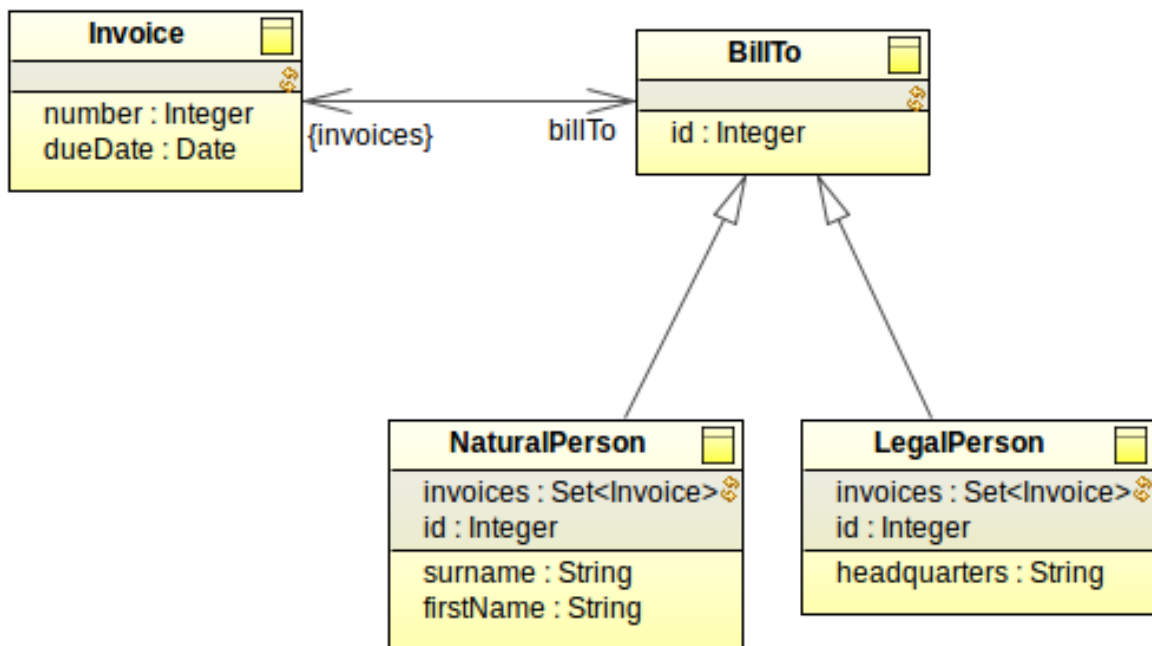
```

5.3 Using Inheritance

Let's rethink our BillTo Record: you could charge a company or a person; a company will not need a surname and a person will not need headquarters data. However, they are still considered the charged parties: to retain this structure, let's create child Records that will allow us to distinguish the charged parties.

Create the child Records *NaturalPerson* and *LegalPerson* of the BillTo Record.

With a bit of luck you have now a model similar to this one:



The BillTo Record is the supertype of the other Records: this means that the child Records have all the fields of the parent automatically.

And there we have another problem: we do not want to create a BillTo record ever again: we want to charge always a natural or legal person. BillTo must be abstract: in the Properties of the Record select the **Abstract** flag.

Now, the assignment of an invoice to John is no longer valid: John must become a NaturalPerson.

```

john := new NaturalPerson (
    name > "John",
    surname > "Doe",
    invoices > {currentInvoice_1, currentInvoice_2}
)

```

For further details on inheritance, refer to the [official documentation](#)

5.4 Defining Methods

As we have mentioned above, Records are similar to classes in object-oriented programming. As such, they can define methods.

Let's define methods for NaturalPerson:

Now define the following methods for the abstract BillTo record:

```
BillTo {
    public Integer getId(){
        this.id
    }
    private void setId(Integer id){
        this.id := id;
    }
}
```

You can now perform the call `john.getId()` which will use the `getId()` method defined on the BillTo record. However, if you try to call the `setId()`, which is a private method on the abstract BillTo record, you will get a validation error.

5.5 Persisting Data

You now have a data type model with a little bit of complexity. But all the data gets trashed right after the model instance finishes. How can you persist the data so it remains available? Change the common records to shared records: an instance of a shared record is stored as an entity and updated instantaneously. So are record relationships.

Let's persist the Records and take a look at what is going on in the database: **Mark all Record as Shared** in their properties.

We got a bunch of errors saying `Shared record <NAME> must specify at least one primary key`. To understand this error, we need to understand that each shared record represents a database table and each record field represents a column in that table, for example, record *Car* with fields *make* and *license plate* is stored in table *Car* with columns *make* and *license plate*. Whenever you create an instance of a shared record, a row with the data is added to the table. Now if you want to find a particular record instance (row), you need to specify something unique; in the example it is the license plate of the car. This unique property is the *primary key*. Now we do not want to create the key manually so set it to be generated automatically:

Note that the parent *BillTo* Record has additional database property O-R inheritance mapping. Its default setting is *Each record to own table*. Let's take a look at what that means:

1. Start PDS Embedded Server and upload the module.
2. Open a database client, such as Squirrel or DBeaver, and check the schema of the Record tables.

For PDS Embedded Server, connect to `//localhost/.h2/h2;MVCC=TRUE;LOCK_TIMEOUT=60000`; for jdbc the URL will be `jdbc:h2:tcp://localhost/.h2/h2;MVCC=TRUE;LOCK_TIMEOUT=60000`. Both the name and password are `lsp`.

You will see that the tables for individual Records reflect the model: *BillTo* table contains only the *id* column; *NaturalPerson* contains *id* identical to the *BillTo.id*, *surname*, *firstname*, etc.

Let's test the other inheritance mapping:

1. Stop the server.
2. Reset the database or change the schema update strategy to Drop/create.
3. Set the Single table per hierarchy setting on the BillTo and Invoice records.
4. Start the server, upload the Module, and check the schema:

The **BILL_TO** and INVOICE table now contain the entire hierarchy along with the TYPE_ID column which holds the record type of the entity, such as invoice-types' NaturalPerson: there are no tables for the *NaturalPerson* or *LegalPerson* records.

5.6 Changes of a Data Model with Shared Records

So you have your data model with your business data, in our case invoicing data. And you decide that you want to store invoicing data in a more organized way: So you remove the dueDate field from Invoice and create a new *BasicData* record with the dueDate field and create a relationship between the two. This is fine if you can just drop the database but what about if you need to preserve the existing data? You cannot simply upload the model since if you force the update you will lose the data on dueDate in Invoice records.

Such cases require manual migration of the database schema, which is an advanced topic and is not covered here (refer to the [developer documentation](#) and [deployment instructions](#)).

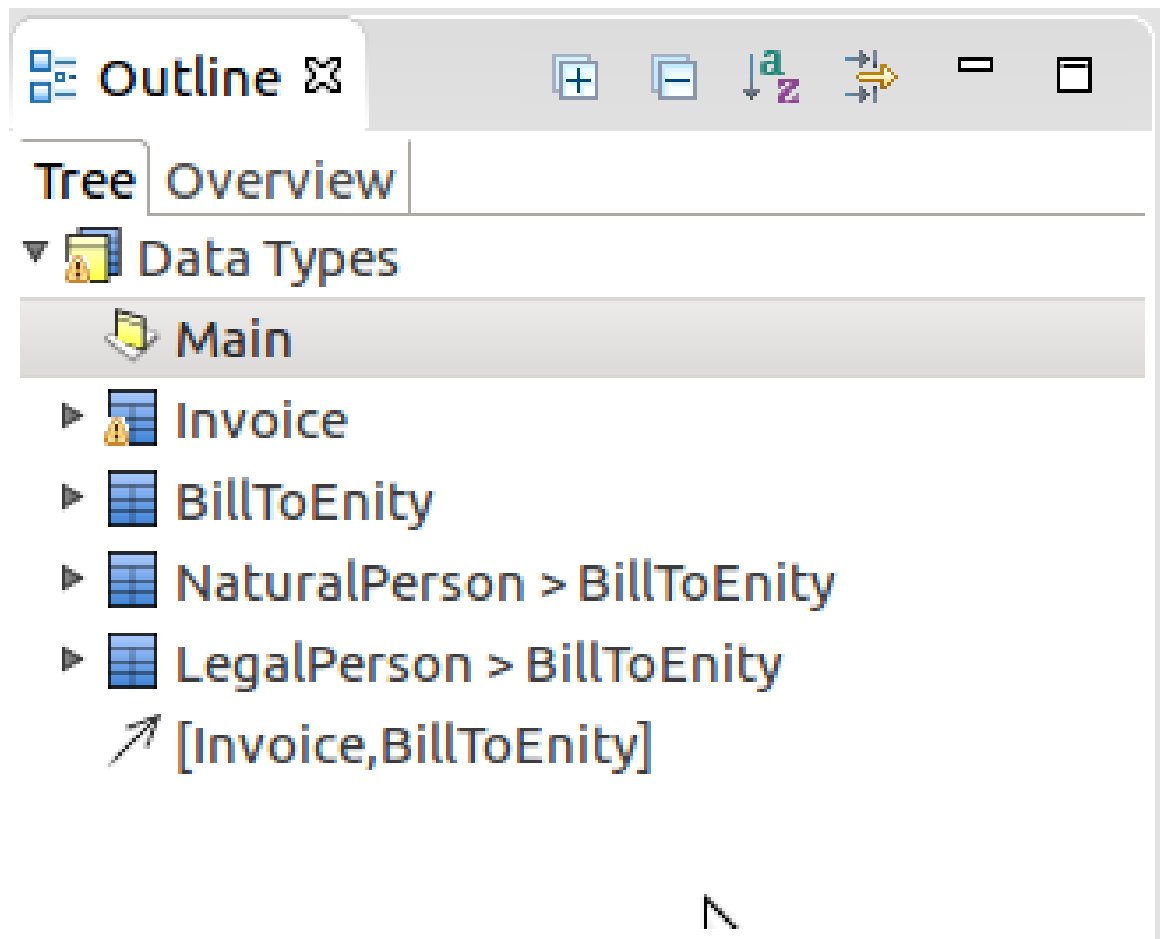
Therefore it is recommended to use a tool for tracking of database schema changes, such as, Flyway or Liquibase even though you might not need it for a very long time.

5.7 Using Diagrams to Organize Records

Until now we have created all elements of our data type model in the default **Main** Diagram. When we inserted an element into the canvas of the diagram, the element was displayed in the diagram and created in the definition file.

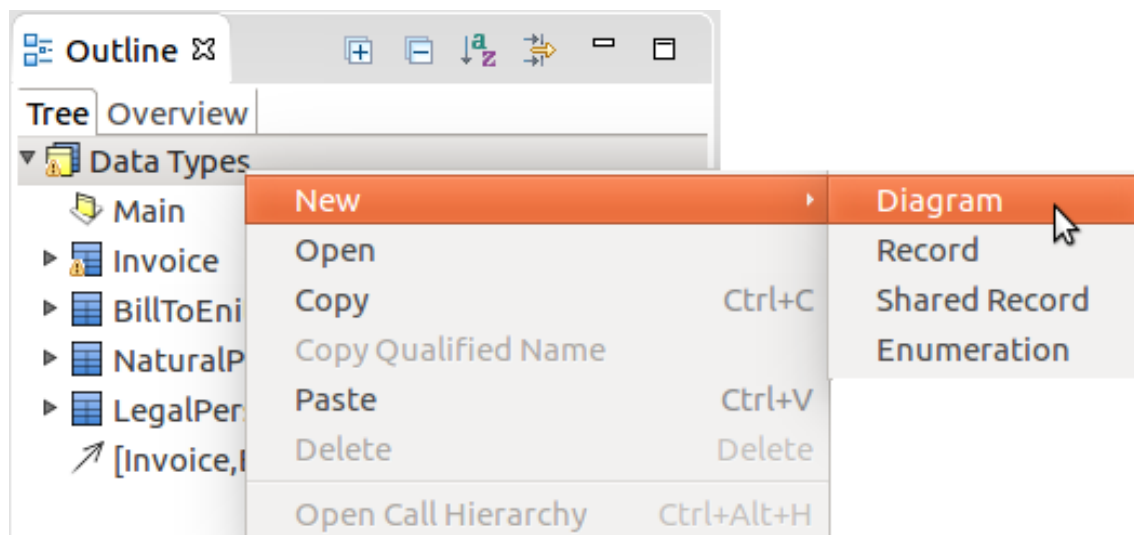
Note that diagrams do not represent a namespace: they are merely a presentation tool. If you create two Records with the same name in two diagrams, this will result in a name clash.

The content of the definition file is displayed in the *Outline* view.



Create another diagram in your definition file and display an existing Record in it:

1. In the *Outline* view, right-click the root *Data Types* node and then **New > Diagram**



2. In the Properties view, enter the name of the diagram, for example Inventory. You could rename the Main diagram to Invoicing so it is easier to work with your diagrams.
3. There is a good chance that you might want to create a relationship between items in Inventory and invoices: since we want to have the inventory-related Records in the new diagram, you want to display that Record also in this diagram: drag-and-drop the Record from the Outline view onto the canvas.

For more information, refer to [the Diagram chapter of the GO-BPMN Modeling Guide](#).

5.8 Using Records from Other Definitions

To display an element from another data type definition file, possibly from a definition file in an imported Module, use the Record Import element, which is available in the palette of the editor.

5.9 Extending the Data Type Model

Continue to further develop the model:

- Declare the `getId()` method on the `BillTo` record and implement it on the `NaturalPerson` and `LegalPerson` records.

Example abstract method declaration

```
public abstract Integer getId();
```

Note that if you declare an abstract method on a Record, you will have to flag the Record as abstract as well.

- Create the `interface` `Nameable` with the `getName` method; make the `NaturalPerson` and `LegalPerson` realize the interface.
 - Create a new Record `InvoiceItem` and make it realize the `Nameable` interface. Add a field of this type to the `Invoice` Record.
-

Chapter 6

Interaction With Users

While the main ambition of BPMN and GO-BPMN processes is to automate your processes as much as possible, there is a good chance that your processes will now and then require input from a human user.

You can get such user input, either as part of a process or at any point in time: depending on this, you will use either a to-do or a document:

- A **to-do** serves to get input for a process.
- A **document** serves to get input at any point in time; no process execution is required; We will demonstrate both options.

6.1 Getting Input as Part of a Process

To get input at a particular moment during a process execution, we need to add a *user task* to the process flow: when the execution reaches the user task, it generates a to-do. The to-do appears in the **To-Do List** in the *Application User Interface* of all *performers*; we say that the to-do has been assigned to the initial assignees.

When one of the assignee performers opens the to-do, the to-do disappears from the to-do lists of other assignees: it is locked by the user. This user interacts with to-do content and once happy with the result they submit the to-do. On to-do submit, the execution of the user task finishes and the execution of the process continues.

Hence, a user task must define:

- the set of its *performers*,
- the content that is displayed when one of the perspective opens the to-do.

Note: In your custom Application User Interface, you will most likely substitute the default **To-Do List** with your custom implementation to display additional information about the to-dos. Instructions on how to do so are available in a **dedicated tutorial**.

Now, we will create a process with a user task, define a simple content for the to-do and demonstrate, how the end-user submits the to-do.

6.1.1 Creating a Process with a User Task

To create a process with a user task, do the following:

1. Create a process definition file.
2. Insert a user task.

We have now a process with a user task: there are multiple errors on the task since we have not defined its required parameters, yet; we will do so shortly.

Note that you do not need to insert a None Start Event or Simple End Event into the process; they are implied.

When the system creates a model instance with this process, the execution will immediately start the user task, which becomes **alive** and starts its execution: it generates a to-do.

6.1.2 Defining To-Do Properties

When the *user task* becomes *alive*, the system generates a to-do with some content for the initial performers or assignees. Hence we need to define the content and the performers:

1. Let's first define the content:

- (a) Create a form definition file.

Important: When creating the form definition, make sure the flag **Use FormComponent-based UI** is *NOT SELECTED*: PDS comes with two different implementations of forms; the flag defines which of the two is used. We are using the *event-driven ui implementation*.

- (b) In the form, insert a button and define on it a listener that will submit the form when admin clicks the button.

2. Now, let's define the properties of the user task:

- (a) Select the user task on the canvas or in the Outline view.

- (b) Focus the Properties view.

- (c) In the Properties view, open the *Parameters* tab and define the following parameters:

- Define the title of the todo, for example, `title -> "Submit Me"`
- Define the admin user as the sole performer in the task properties: `performers -> {getPerson("admin") }`
The `getPerson()` function call returns the **admin** user: no other user will see the to-do in their to-do list..
- Define the form you created as the to-do content: this is done in the `uiDefinition` parameter of the user task, for example, `uiDefinition -> submitForm()`.

After you save changes, your project should be error-free.

6.1.3 Running the Model

Let us run the model: start PDS Embedded Server.

The system created a model instance of our model and triggered the execution: it is stuck on the User task and waiting for the submit from the Admin: open the Application User Interface as the admin user and submit the to-do.

When you as the admin user clicked the Submit button in the to-do, the execution flow continued:

- in our case the to-do ceased to exist,
- the user task became finished; because there was no execution going on in the process instance,
- the process instance finished and subsequently also the model instance.

You can check the status in the Management perspective.

6.2 Getting Input Outside of a Process

If you want to create a page that will take input from a user at any time, create a document: similarly to a to-do, its content is a form. Unlike a to-do, the document is accessible as long as the module with the document is in the Module Repository of the server: Every user has access to the Documents they are entitled to at all times.

Note: When you submit the document, the document remains accessible on the Documents page↔: under the hood, whenever a user opens a document, the system creates a model instance with the document. This instance contains only an instance of the document and finishes when the user submits the document.

We will now create a document with the same content as the to-do (we will use the same form).

6.2.1 Creating a Document Definition

To create a document, do the following:

1. Create a document definition file with a new document definition with the following properties:
 - UI Definition: the form with the submit button, for example, `submitForm()`
 - Access right: access rights of the user to the document; set it so that any user can access the document; that means it must always evaluate to **true**
 - Navigation: page to go to after the document is submitted
By default, the user is redirected to home page. This property overrides the home redirect. You could set it to the Documents page:

```
{ s:Set<Todo> -> new AppNavigation(code -> "documents") }
```

6.2.2 Uploading and Accessing a Document

Run or connect to your server and upload the module with the document.

Now you can open the document as any user.

Chapter 7

Goal Hierarchies

In LSPS, you can create BPMN processes just the way you know them; however, LSPS provides support for goal-based process modeling, extension of BPMN.

Goal-oriented processes allow you to create unstructured processes, where you separate what your business process should achieve from how it should achieve it:

- what to achieve is defined in goal Hierarchies
These are designed primarily as trees of achieve goals and plans.
- how to achieve the required state is defined in plans

Let us just dive in and demonstrate the goal-oriented approach on an example.

7.1 Defining Your Goals

Let us consider a process for arranging a business trip:

- The main goal or purpose is to have the "Trip arranged".
- The trip is considered arranged after the accommodation, flights, and transport to and from the airport is arranged.

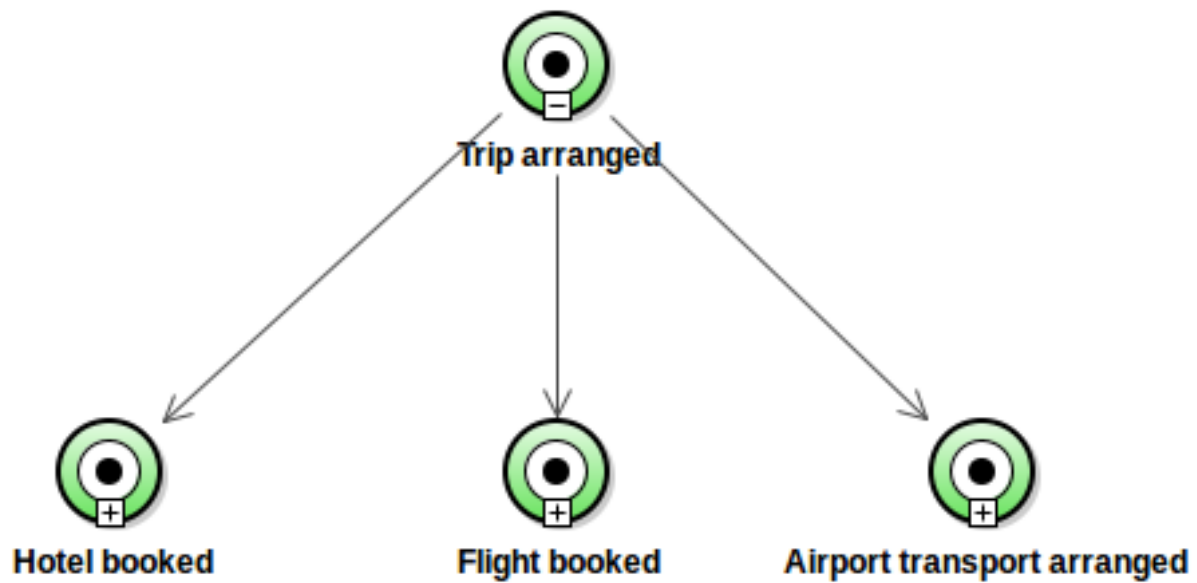
Once the trip is arranged, the purpose of the process was met: this is the main goal.

So here we identified the main goal to arrange the trip and the 3 sub-goals it comprises. In GO-BPMN, the goals are represented by Achieve Goals arranged in the required hierarchy.

In this case, the main goal will be a top Achieve Goal, that is, an Achieve Goal with no incoming "arrows" or decompositions: such goals are started, or triggered, by the process: when the process starts, the top goals start.

To achieve the goal we must achieve three things: these are subordinate to the main goal. The sub-goals are represented again by Achieve Goals. This time they are connected to their main goal by decompositions: these goals will be triggered by their main goal at once: Goals are executed in parallel manner.

The Goal hierarchy of the process will hence look like this:



Now, let's create the hierarchy.

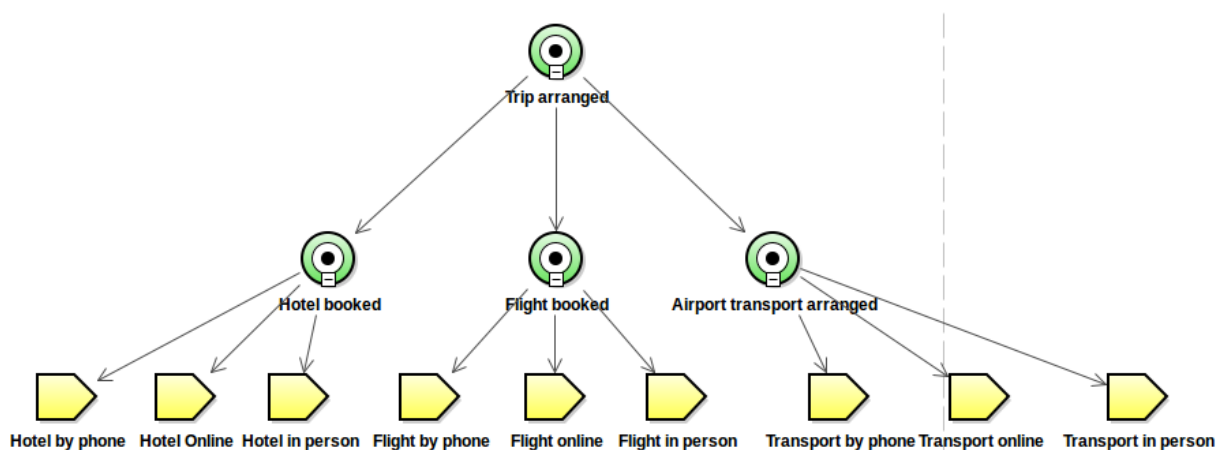
7.2 Defining How to Achieve Your Goals

We have defined what we want to achieve. Let us get to the how-to-achieve part. This is defined by Plans.

Each goal can be achieved in one or multiple ways: We can book a hotel, tickets, and the airport transportation in different ways, for example, by phone or online.

Generally, it does not matter how we achieve it: Therefore, a goal can have multiple Plans. The inside of a Plan defines how to achieve the task ahead using the standard BPMN flows that perform the action.

Now, decompose the sub-goals into plans. You should end up with a hierarchy similar to the one depicted below.



How to achieve a Plan or a BPMN process is defined by a standard BPMN flow in the Plans or BPMN Processes.

Let's start by defining a dummy flow in one of the Plans.

This flow simply starts the plan execution and immediately finishes: the None Start Event produces a token which passes through the Flow to the Simple End Event, which consumes the token. In your processes, you will want to execute Tasks, split your flow with Gateways, etc.

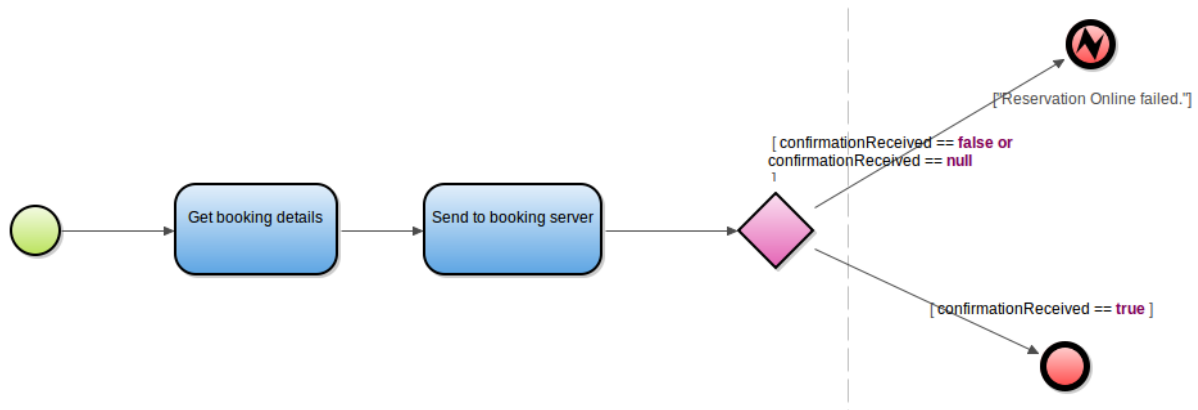


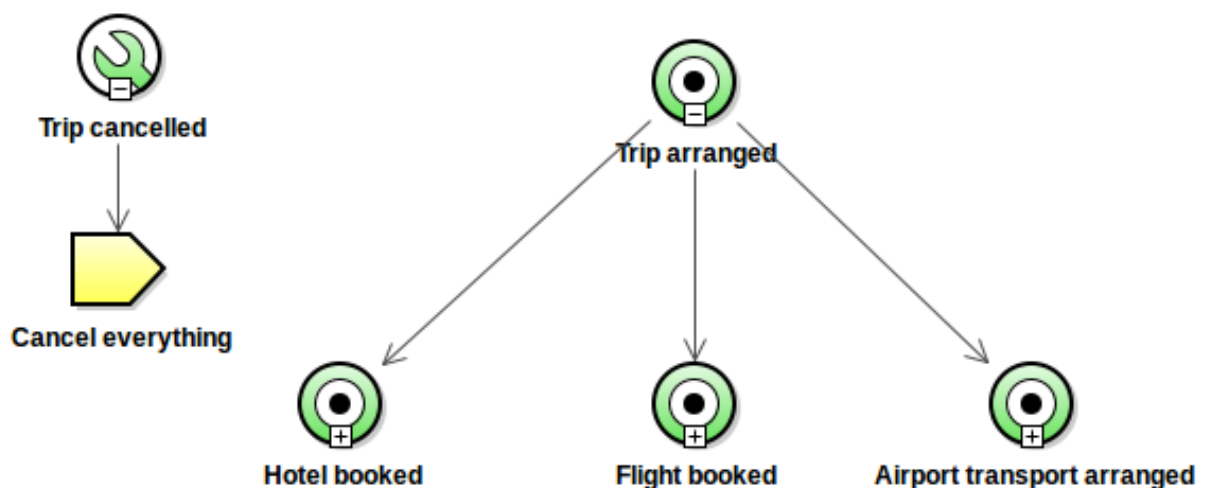
Figure 7.1 Example of a Plan body

7.3 Making Sure Things are on Track

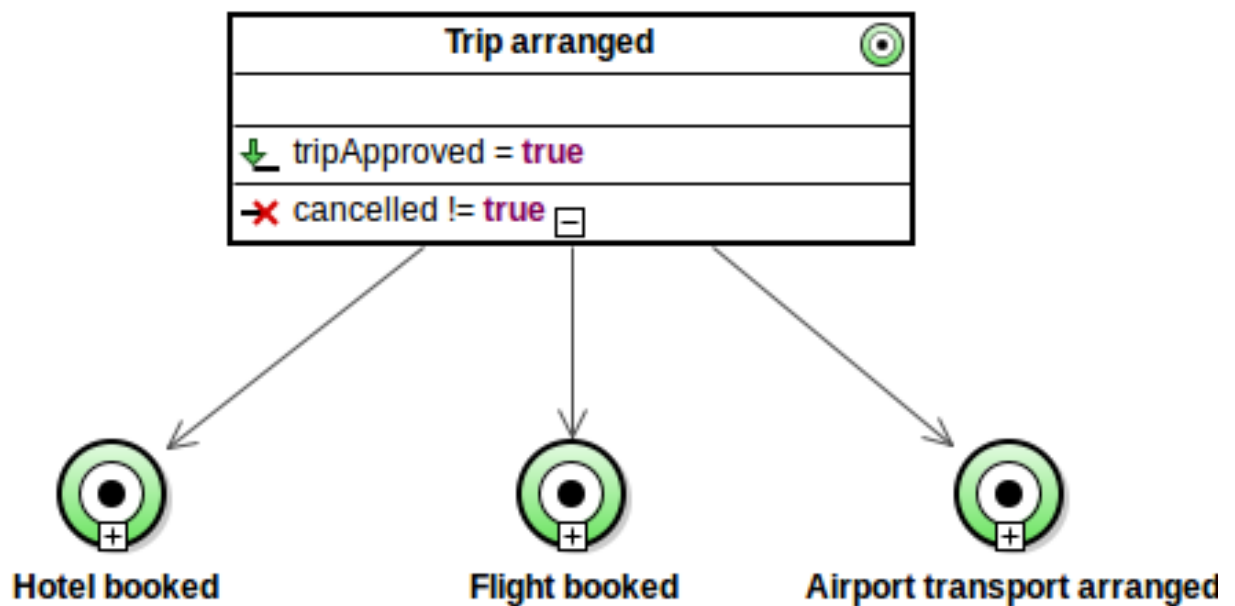
Some Goals or the entire Process might need to run only under certain conditions, for example, if the business trip gets cancelled, it does not make any sense to continue the execution. You might even need to rollback what was already arranged.

You can deal with conditions in goal hierarchies either using a Maintain Goal or with preconditions and deactivate conditions of Achieve Goals:

- If you want to rollback or compensate some action, use Maintain Goals: these goals define a condition: the moment the condition becomes false, the Maintain Goal triggers one of its Plans.



- If you need to check the condition before something happens, use pre-conditions of Achieve Goals.
- If you need to interrupt an Achieve Goal when something happens, define its deactivate condition.

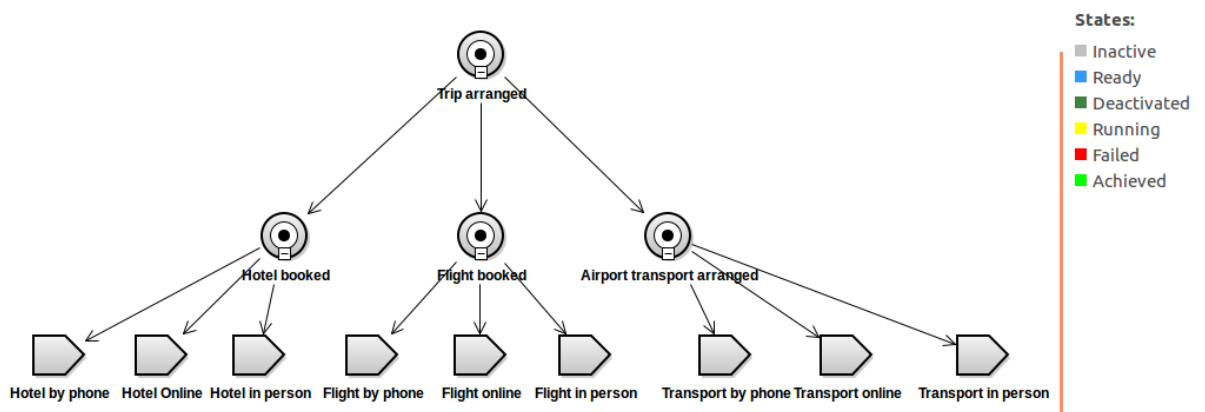


7.4 Execution

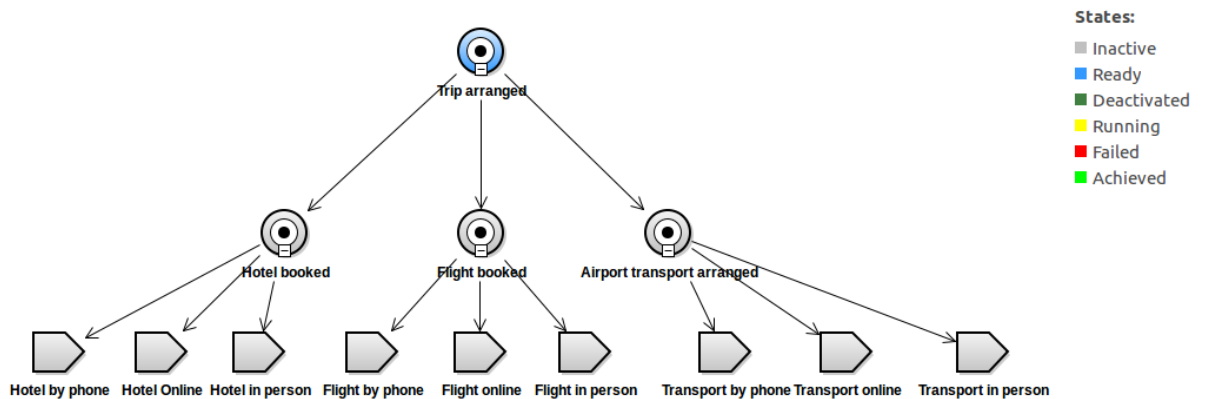
Let us take a look at the execution of a goal hierarchy and its process: note that the goal hierarchy is part of the process.

When you instantiate the model, the following happens:

1. One process instance is created for each process: the context data is created (variables are created and have their initial values assigned).
Note that only processes that have the flags Executable and Instantiate Automatically are instantiated immediately.
2. All Goals become inactive.



3. Top Goals become *Ready*.



4. Top Achieve Goals with their precondition evaluated to *true* become **Running**. If there is at least one such Goal, the process instance becomes *Running*.

- If the running Goal is decomposed to sub-Goals, the sub-Goals become *Ready*. These then continue their life cycle (have their precondition checked, etc.). The goal is achieved, when all its sub-Goals become *achieved*.

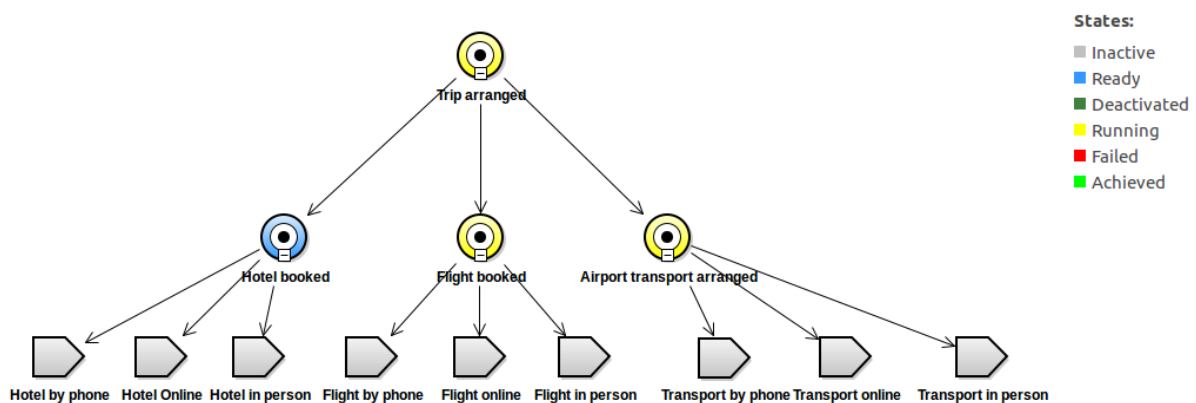


Figure 7.2 Triggered Subgoals

- If the running Goal is decomposed to Plans, the Plans have their conditions checked and **one Plan** with the condition *true* becomes *Running*: the None Start Event of its body is triggered.
 - If the body **ends with an Error End Event**, or it receives an error event, the Plan finishes as **Failed** (note the Plan must define the code of the error in its property Failure error codes) and the Goal triggers another Plan that was not executed yet.

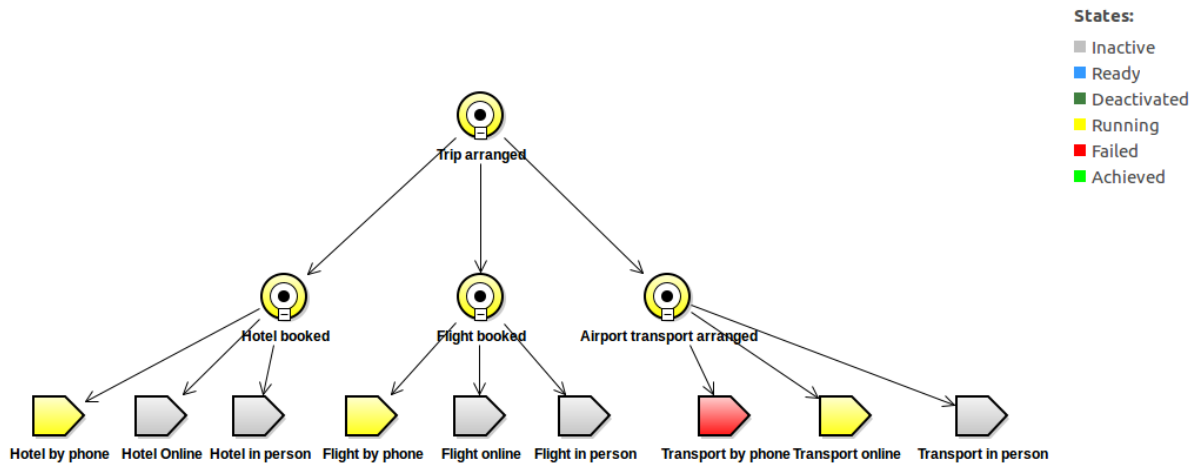


Figure 7.3 An alternative Plan running after a Plan failure

- When the body ends with **another End Event**, the Plan finishes as **Achieved**.
- When a Plan finishes as **Achieved**, its Goal becomes *Achieved*.

5. Once all Achieve Goals are *Failed* or *Achieved*, the process instance becomes *Finished*.

7.5 Summary

Goal-oriented approach to processes makes only sense if there are multiple parallel actions required to meet the goal.

If you want to use the goal-based approach, make sure the business process analysis is performed accordingly↔: the "extraction" of goals requires a change in the mind set. However, this additional effort can result in more autonomous business processes.

When designing such a process, you should proceed as follows:

- Extract the purpose of your process into Goals.
- Identify the possible ways of how to achieve the leaf Goals of your hierarchy.
- Decompose the leaf Goals into Plans and design the Plan bodies.

7.5.1 Pros

- Dynamic flow execution: Some goals might never be triggered or triggered only after an event.
- Process instance restart based on conditions if Plans are well-structured, that is, they are minimalistic.
- Performing multiple process parts synchronously.

If your model does not require any of the above, consider using plain BPMN.

7.5.2 Cons

- Business Process Analysis of goal hierarchies might involve a steeper learning curve than that of BPMN processes.
- Only one Goal hierarchy is instantiated per process instance (no multi-instance Goals).
- Goal hierarchies do not support processing of BPMN events since there are other mechanisms that substitute these.
- Goal hierarchies are not legible if they contain conditions: the semantics are not obvious from the diagrams.
- If hierarchies have many Goals with conditions that are checked continuously and contain execution, for example, assignment expressions, they can result in performance issues.

7.6 Documentation

- Achieve Goals
 - Maintain Goals
 - Plans
-

Chapter 8

Putting It Together

We are going to create an example model that will register clients: A client will log in to the application, enter their registration data, and submit the data. An agent will accept or reject the registration data.

We have to consider the following:

- *Data structure:* The specific business data involved is the registration data of the user. The data must remain available once provided: we will therefore store it as a shared Record.
- *People involved:* Two types of users, the client and the agent, are involved in the process. We will need two roles that will represent these user types.
- *Processes:*
 - The registration must be available to clients at all times and does not involve any decision-making logic or work distribution; hence no BPMN process is required. We will implement the registration as a Document available to all client users.
 - The approval request requires distribution of work: the request should be available to all agents after a client submits their data at first. Therefore, we will implement the approval as a process triggered when a client submits the registration.

8.1 Prerequisites

Before we can start implementing the solution, we need to create a GO-BPMN project:

1. Make sure you are in the Modeling perspective: GO-BPMN Explorer should be displayed on the left.
If this is not the case, go to Window > Perspective > Open Perspective > Other on the main menu and select the Modeling perspective in the dialog.
2. On the main menu, go to **File > New > GO-BPMN Project**.
3. In the **New GO-BPMN Project** dialog, enter **ClientRegistration** as the project name and click **Finish**.

8.2 Registration Data

We want to work with client's registration data, a custom data structure comprising multiple pieces of information; to store such data, we use Records. Since we want the data persisted so they remain available after the model instance has finished, it must be a *shared* record. This is the only piece of business data we need. Therefore, our data model will contain a *sole* *ClientRegistration* *shared* record.

Since we might need to use the data model in other models in the future, such as, regular user-data review or user-data update, we will create the data model in a dedicated module so we can reuse it:

1. Right-click the *ClientRegistration* project and click New > GO-BPMN Module.
2. In the dialog, enter the module name **common** and unselect the **executable module** option (it makes no sense to create a model instance over a module with a data model).
3. Click **Finish**.

Now create the *RegistrationData* record:

1. In the *common* module, create the data type definition file: right-click the *common* module, go to New > Data Type Definition and click **Finish**.
2. In the editor on the right, create the **RegistrationData** shared Record.
3. Add the following fields to the Record:
 - **approved**: Boolean
 - **firstName**: String
 - **lastName**: String

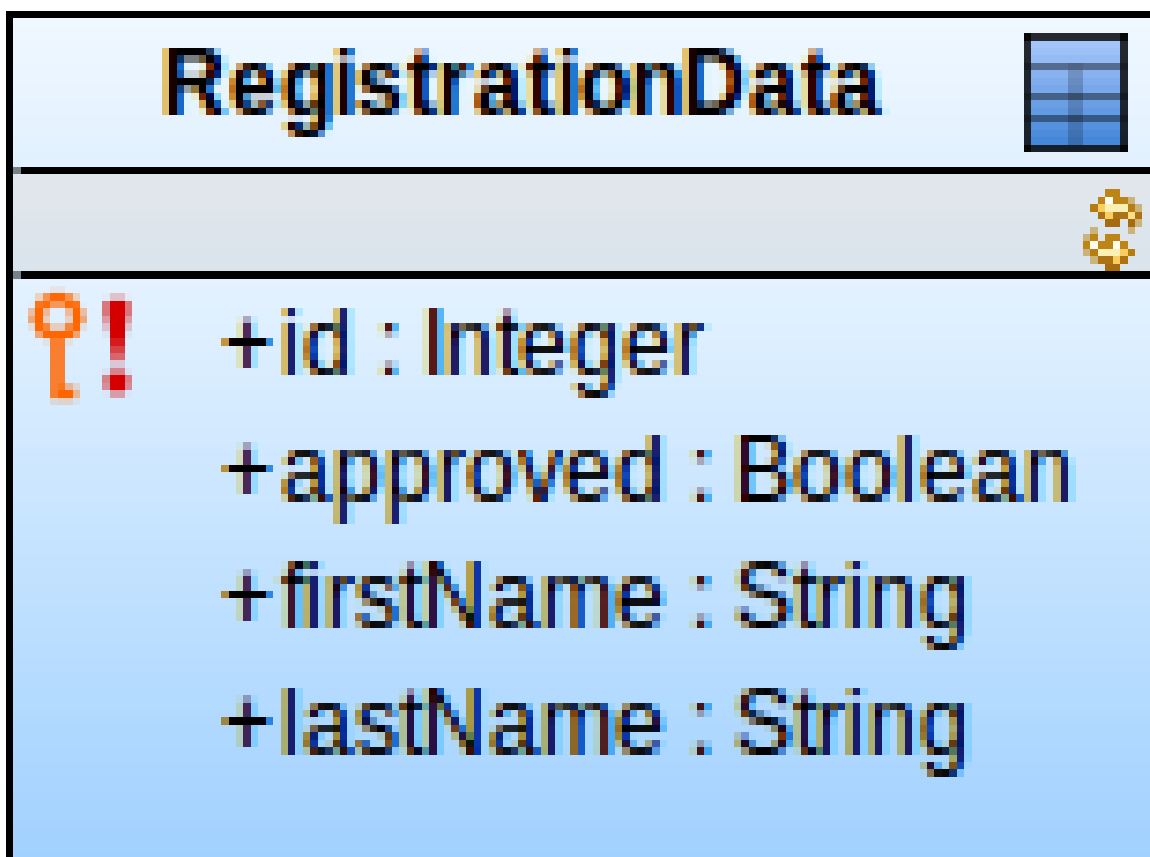


Figure 8.1 RegistrationData record

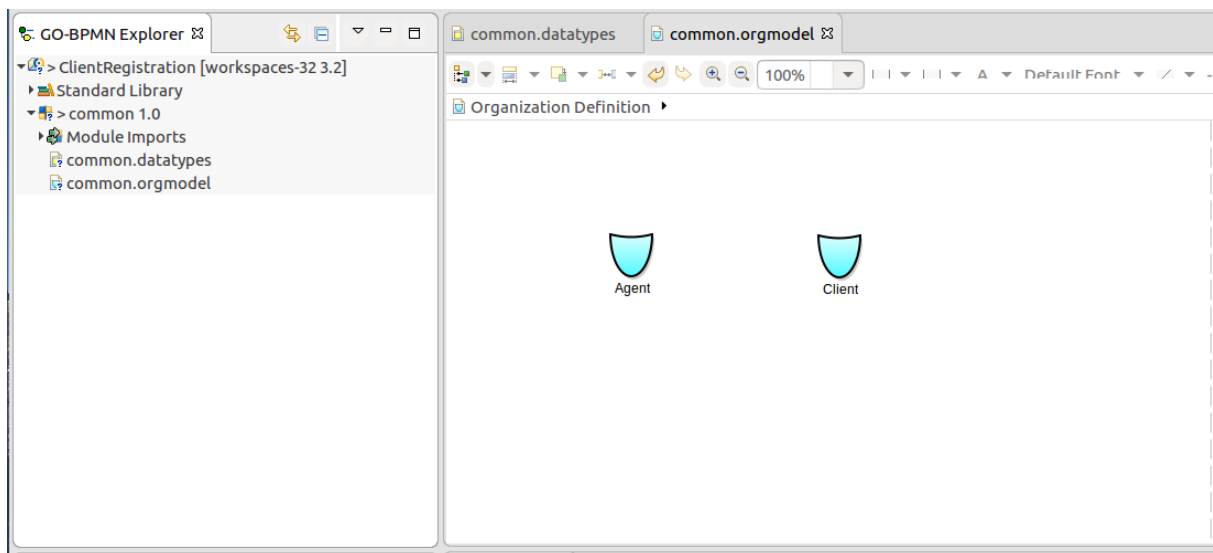
8.3 Client and Employee User

In our registration process, we are considering two types of users: The *client* who enters new registration data and the *agent* who approves the registration. To distinguish between the users, we will use roles: Roles allow us to group users with the function.

Roles are created in an organization model. Generally, the model should not attempt to reflect your real organization structure but the hierarchy of users involved in the business process. Detailed information on how to design and use organization models is available [here](#).

Let's create the organization model:

1. In the *common* module, create the organization definition file: *right-click the common module*, go to **New > Organization Definition**.
2. Create the roles **Agent** and **Client**.
3. Save.



Now you can use calls `Agent()` and `Client()` to get all users with the given role.

8.4 Registration and Approval

The registration and the approval processes together constitute the complete registration. They are no longer static data, like the data model and organization model, and required involvement of user and changes in time—they need to be instantiated and executed.

Therefore they will be stored in an executable module so we can create model instances over them:

1. Right-click the project and click **New > GO-BPMN Module**.
2. In the dialog, enter the module name **registration** and select **executable module**.
3. Click **Next**.
4. Click **Add** and double-click the *common* module to import it into the registration module.

If a module is not executable, it is instantiated only when imported into another executable module: it is a module instance of a model instance that was created by another module.

8.4.1 Registration Process

When a client wants to register, they open the registration form, fill it out, and submit it. All is performed on a single page that is always available and does not require any further execution logic: no timer or event triggering of actions, no involvement of other users, etc. Hence we can implement it as a document.

1. In the *registration* module, create a document definition with a *registrationDocument* document with the following properties:

- Name: `registrationDoc`
- Title: `"Registration"`
- UI definition: `clientRegistration()`
- Access rights: `isPersonIn( getCurrentPerson(), Client() )`

Note that we have not created the UI definition with the document content, yet (There is an error reported on the UI Definition expression).

Let's create it:

1. Create the form definition file: right-click the *registration* module, go to **New > Form Definition**.
2. In the dialog, enter the name of the file `clientRegistration`.
3. Make sure the **Use FormComponent-based UI** option is *not* selected (It activates the *experimental forms* version, in which you can create forms in a more code-like way).
4. Click **Finish**.
5. In the form, create a form variable that will hold the registration data:
 - (a) Right-click the *clientRegistration* node in the Outline view and go to **New > Variable**.
 - (b) In the properties of the variable set:
 - Name: `newRegistration`
 - Type: `RegistrationData`
 - Initial value:


```
new RegistrationData(approval -> false)
```
6. Design the form content:
 - (a) Insert a vertical layout, then a form layout into it;
 - (b) Into the form layout, insert Text Boxes for the `firstName` and `lastName` fields and do the following in their Properties view:
 - i. Insert the name in the Label field.
 - ii. Set its Binding to the field of the `newRegistration` variable, such as `&newRegistration.firstName`.
7. Create the register button and define the register action:
 - (a) Insert the Register button.
 - (b) In its Properties view, go to the Event Handling tab and click the Add button in the Private Listeners section.
 - (c) Go to the Actions tab and select the Submit option: this will make sure, that the data in the form will be saved and the form discarded when the user clicks the button.

8.4.2 Approval Process

Now we need to create the approval process for the agent:

- The process starts when the client clicks the *Register* button.
- It displays to an agent a form with the user name, and an approve and reject button.
- It stores the data when the agent submits the form.

1. First, implement the Register button logic:

- In the *clientRegistration* form, open the ActionListener of the Register button.
- On the Actions tab, select Persist to enable the section below: here you can define an expression that will be executed upon Persist: we want to create a model instance of our module so it will run the approval process we will create in the next step:

```
createModelInstance(
    synchronous -> true,
    model -> thisModel(),
    processEntity -> newRegistration,
    properties -> null
)
```

2. Create the process definition file for the *approval* process:

- Right-click the *registration* module, go to **New > Process Definition**.
- In the dialog, enter the name of the process **approval**, select the **BPMN-based process** as its type.
- Leave the *Executable* selected.
If you unselect the Executable option, the process cannot be executed: this option is useful for documentation processes.
- Leave the *Instantiate Automatically* option selected.
We want to instantiate the process automatically when we create the model instance. If you unselect the *Instantiate automatically* option the process instance will not be created and triggered when the model instance is created; such processes are useful when they act as **reusable subprocesses**.

3. Now design the process:

- Insert the None Start Event from the palette.
- Drag-and-release the quicklinker of the None Start Event and in the context menu, select the **Task**.
 - You will be prompted to select the type of the task: our task should require a user action: select the **User** task.
- Define the task parameters on the Parameters tab in the Properties view:


```
title /* String */ -> "Client Registration Approval",
performers /* Set */ -> {Agent()},
uiDefinition /* UIDefinition */ -> registrationApproval()
```
- Attach the Simple End Event to the *GetApproval* task so the process finishes when the user submits the approval.

4. Create the *registrationApproval* form:

- Create a new form definition.
- In the form, create a form variable that will hold the registration data:
 - Right-click the *registrationApproval* node in the Outline view and go to **New > Variable**.
- In the properties of the variable set:
 - Name: registrationRequest
 - Type: RegistrationData

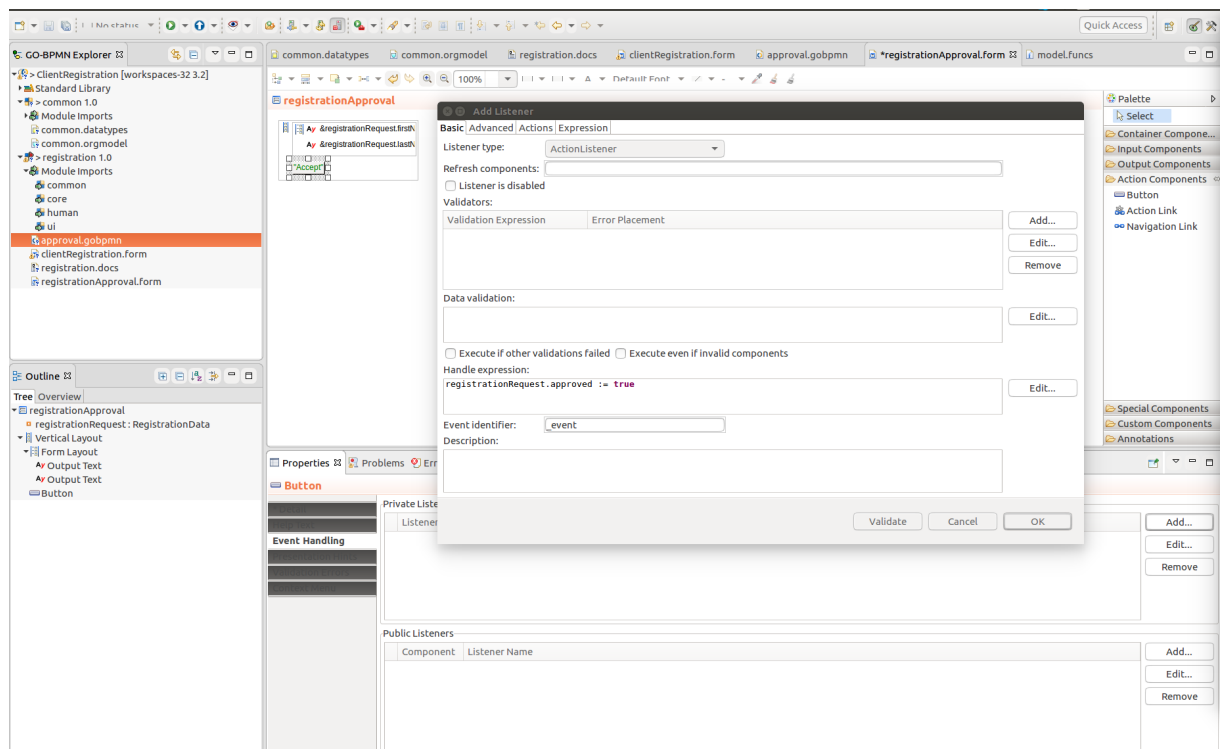
- Initial value: initialize the value to the model entity (the entity is passed by the `createModelInstance()` call of the Register button):
`getProcessEntity(RegistrationData)`

5. Design the form content:

- Insert a vertical layout, then a form layout into it.
- Into the form layout, insert Output Texts for the `firstName` and `lastName` fields and set their Binding as Reference to the field of the `registrationRequest` variable.

6. Insert the *Accept* button and set its Binding to `®istrationRequest.approved` variable

- In its Properties view, go to the Event Handling tab and click the Add button in the Private Listeners section.
- On the Basic tab, set as Handle expression `registrationRequest.approved := true`



- Go to the Actions tab and select the Submit option: this will persist the data and submit the task.

7. Insert the *Reject* button and set its Binding to `®istrationRequest.approved` variable

- In its Properties view, go to the Event Handling tab and click the Add button in the Private Listeners section.
- On the Basic tab, set as Handle expression `registrationRequest.approved := false`
- Go to the Actions tab and select the Submit option.

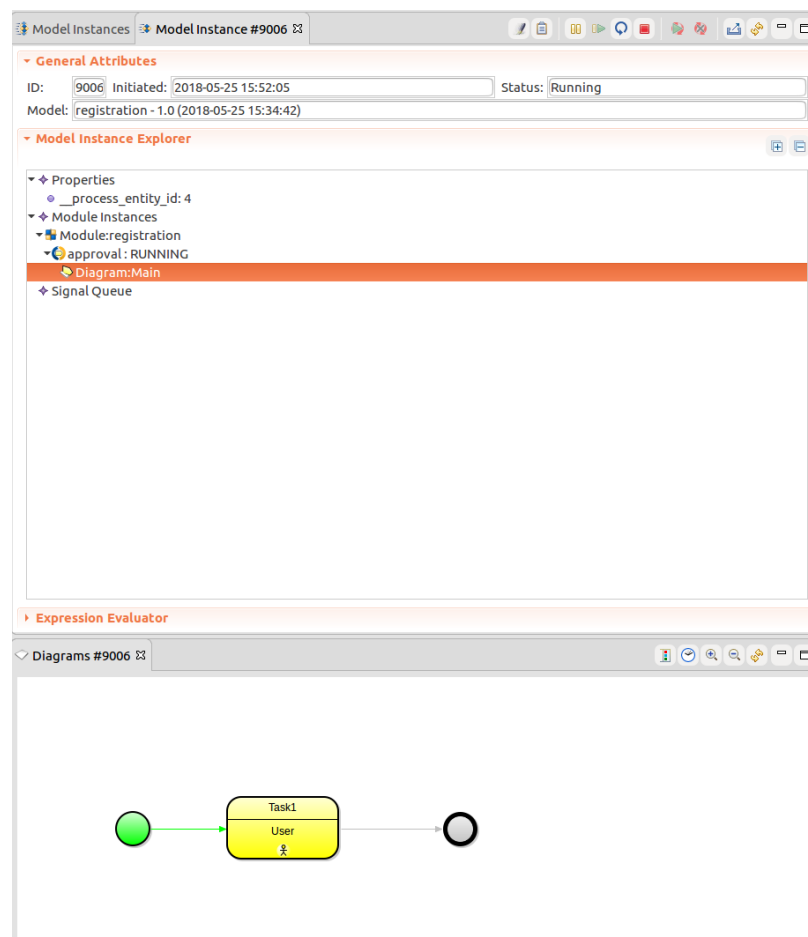
8.5 Running the Model

Now upload your model, create its instance, assign the Roles to users and follow the execution:

- Connect PDS to an LSPS Server to which you want to upload your model: Consider using the **PDS Embedded Server**, which runs locally and is stored in your workspace.

You can check if your PDS is connected to an LSPS Server in the status bar at the very bottom of PDS.

2. Upload the model: In the GO-BPMN Explorer, right-click the *registration* module and select **Upload As > Model**: now, your module is in the LSPS Server Module repository.
3. Assign the Client role to the admin user.
4. Log in to the Application User Interface as admin and submit the Registration document.
5. Back in the Management perspective, refresh the Model Instances view: a new model instance of registration appears. This was triggered by the `createModelInstance()` of the Register button.
6. Double-click the model instance: in the detail view, expand the model tree and double click the approval process diagram.



7. The process instance is stuck on the user task: the task generated a to-do for an end user with the role Agent; however no such user exists: Assign the Agent role to the guest user so you can submit it.

The screenshot shows the 'Model Instances' application interface. At the top, there's a tab for 'Person: guest'. Below this, the 'User Information' section contains fields for Login (guest), Password, Password confirmation, and State (Enabled). The 'Application Data' section contains fields for First Name, Last Name, Email, and Phone. A modal window titled 'Modeled Roles for guest:' is open in the foreground. It has a 'Module' dropdown set to 'common', a 'Role name filter' set to 'Agent', and a 'Filtered roles' dropdown also set to 'common::Agent'. Below these is a table with columns 'Parameter' and 'Value'. To the right of the table is a 'Resulting selection:' list with two items: 'common::Agent' (checked) and 'common::Client' (unchecked). There are 'Select All' and 'Deselect All' buttons next to the list. At the bottom of the modal are 'Cancel' and 'OK' buttons. In the background, a 'Details' section is partially visible with a table with columns 'Key' and 'Value'.

8. Log in to the Application User Interface as *guest* and submit the approval todo under the Todo menu.

Now go to the Management perspective, refresh the diagram of your process instance: the instance finished and, since there is not other process instance running, also the entire model instance finished. Note that the registration data remained persisted in the system database and you could use it in another model instance.

Chapter 9

Customizing the LSPS Application

In the previous chapters, you uploaded your modules to the PDS Embedded Server. This is an application server with the LSPS Application already deployed. The server runs on your computer locally and its LSPS Application cannot be modified.

In real-world scenarios, such a solution is obviously not sufficient since it is not possible to customize the LSPS Application: You cannot change the look of its Application User Interface or add your custom business logic.

To perform such customization, you need to modify the [LSPS Application](#) and deploy it to a supported application server (to set up such an application server, refer to the [deployment instructions](#)).

Important: Before you start customizing the LSPS Application, analyze your business processes thoroughly.

9.1 Generating the LSPS Application

To generate the maven archetype of the LSPS Application and related resources, you need [LSPS Enterprise Edition with SDK installed](#): from a PDS with SDK, you can [generate your application](#). The maven archetype of the application along with SDK Embedded Server, and launch and build configurations will be created and added to your workspace. Note that LSPS Application is a standard enterprise application: To deploy it on a supported application server, all you need to do is to build the application's sole EAR and deploy it to a supported application server as described [here](#).

Using the generated resources, you can do the following:

- add Java code, EJBs
- add custom task types, functions, form components.
- customize the Application User Interface

Detailed instructions on how to perform such customization are available in the [Development Guide](#).

